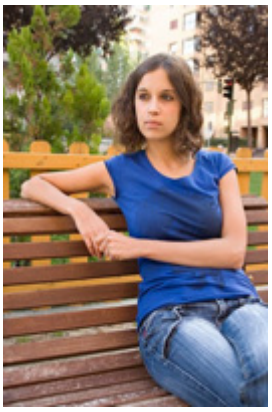
Bases de datos xml.

Caso práctico



A la empresa BK Programación no dejan de llegar nuevos proyectos, está claro que sus productos finales convencen a sus clientes y éstos están bastante contentos con ellos.

Ana está orgullosa de haber participado y aportado su granito de arena en el desarrollo y mantenimiento de diferentes aplicaciones, algunas bastante complejas.

Este fin de semana se ha tomado un respiro. Sabe que el lunes comienza con algo nuevo. Debe ayudar a **María** y **Juan** en la tarea de compatibilizar diferentes formatos de datos que intercambian varias academias de Inglés de una misma firma.

Las academias gestionan la información relativa a sus cursos, alumnos, profesores, distribuidores de material, libros de lectura, artículos, apuntes, exámenes, etc. con programas diferentes y basados también en sistemas de bases de datos diferentes.

Hasta ahora, las academias han hecho uso de XML para intercambiar los datos entre sus sistemas, pero cada vez surge algún problema nuevo, sobre todo con el almacenamiento, consulta y recuperación de ciertos documentos de texto. Por tanto, han decidido acudir a BK Programación para buscar la mejor solución. No les importa empezar desde cero y, si es necesario, utilizar una base de datos nativa XML, tal y como les ha sugerido **Ada** en su primera entrevista.

Ana apura la tarde del domingo pues esa misma noche va a comenzar su nueva aventura, ¡repasar XML y las tecnologías relacionadas! **Ana**, es muy profesional y no deja para mañana lo que puede hacer hoy.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

Materiales formativos de FP Online propiedad del Ministerio de Educación, Cultura y Deporte.

Aviso Legal

1.- Introduccion.

Atendiendo al nivel de estructuración, podemos decir que existen tres tipos de datos:

- **Datos estructurados**. Son los que tienen un formato estricto. Toda la información recogida se ajusta al mismo formato, como por ejemplo: los datos tabulados en filas y columnas de una tabla.
- **Datos desestructurados**. No tienen ninguna estructura, como un documento de texto o un archivo de vídeo.
- **Datos semi-estructurados**. Tienen cierta estructura, pero no toda la información recogida tiene la misma forma, y además puede ir variando de manera dinámica.

Las bases de datos tradicionales, como las Bases de Datos Relacionales (BDR) son apropiadas para almacenar datos estructurados, pero la cuestión es que, en la actualidad, son muchas las situaciones en las que interesa almacenar grandes volúmenes de datos no estructurados, e incluso integrarlos con datos estructurados. Pero ¿cómo hacerlo? Aquí es donde entra en juego XML (eXtensible Markup Language).

XML define un conjunto de reglas semánticas que permiten la organización de información de distintas maneras. Es un estándar definido por el W3C y ofrece muchas ventajas, entre ellas:

- Es un lenguaje bien formado. No puede haber etiquetas sin finalizar.
- Extensible. Permite ampliar el lenguaje mediante nuevas etiquetas y la definición de lenguajes nuevos
- Fácil de leer:
- Autodescriptivo. La estructura de la información de alguna manera está definida dentro del mismo documento
- Intercambiable. Portable entre distintas arquitecturas.
- Para su lectura e interpretación es necesario un parser, y hay productos y versiones libres.

Debido fundamentalmente a estas ventajas y a su sencillez, **el estándar XML ha sido ampliamente aceptado y adoptado para el almacenamiento e intercambio de información**, y como consecuencia de este uso se ha creado la necesidad de almacenar dicha información.

Y ¿cómo se almacena la información en formato XML de cara a su recuperación y consulta? Existen varias aproximaciones para organizar y almacenar información XML que fundamentalmente van a depender de los diferentes archivos o documentos XML que nos podemos encontrar (centrados en datos y centrados en texto).

Pero en definitiva, lo que se busca es una estrategia que permita almacenar y recuperar datos poco estructurados con la eficiencia de las Bases de Datos convencionales, y es por ello que surgen las Bases de Datos XML.

Citas para pensar

El experimentador que no sabe lo que está buscando no comprenderá lo que encuentra

Claude Bernard

Para saber más

En el siguiente enlace puedes consultar diferentes tecnologías basadas en XML.

Tecnologías XML

1.1.- Documentos XML centrados en datos y centrados en texto.

Como te hemos comentado antes, **los tipos de documentos XML que nos podemos encontrar** son:

- **Documentos centrados en datos**, con las siguientes características:
  - Muchos elementos de datos de pequeño tamaño.
  - Con estructura regular y bien definida.
  - Datos muy estructurados o semi-estructurados.
  - Dirigidos a utilización automática (por máquinas).
  - Ejemplos: Facturas, Pedidos Ficha de alumno.
- **Documentos centrados en texto, contenido o documentos** con las siguientes características:
  - Pocos elementos.
  - Con grandes cantidades de texto.
  - Con estructuras impredecibles en tamaño y contenido.
  - Datos poco estructurados.
  - Orientados a ser interpretadas por humanos.
  - Enfocados a sistemas documentales y de gestión de contenidos.
  - Ejemplos: Libros, Informes, Memorias, Artículos bibliográficos.

En la siguiente imagen ampliable puedes apreciar las características indicadas para estos dos tipos de documentos.

En el siguiente enlace tienes un ejemplo de cada tipo de documento.

Ejemplo de documento centrado en datos y documento centrado en texto. (0.02 MB)

Para saber más

Te proponemos el siguiente enlace para repasar el significado de documento XML bien formado.

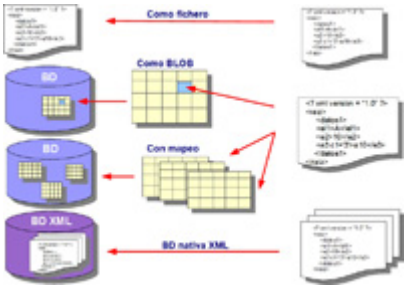
Documento XML bien formado.

1.2.- Opciones de almacenamiento.

Para almacenar documentos XML tenemos las siguientes opciones:

- **Almacenamiento directo del fichero**. Es una opción pobre ya que las operaciones que podemos hacer sobre ellos son limitadas, las que proporcione el sistema de archivos.
- **Almacenar el documento en una base de datos existente** (base de datos relacional, orientada a objetos u objeto-relacional). En este caso existen las siguientes posibilidades:
  - **Directamente como una columna tipo binario grande (BLOB) dentro de una tabla**. En este caso se almacena el documento intacto. Es una buena estrategia si el documento XML contiene contenido estático, que solo será modificado cuando se reemplaza el documento completo. Almacenar el documento completo en formato de texto es fácil de implementar dado que no se necesita mapeo o traducción, pero limita las consultas y búsquedas de contenido.
  - **Mediante mapeo basado en tablas, o basado en objetos**. En ambos casos hay que realizar una transformación para ajustarlo a la estructura de la BD antes de almacenarlo, y normalmente esto conlleva prescindir de cierta información, que supondrá que el documento y su formato no se almacena y recupera de manera intacta. Además, en el caso de documentos centrados en texto, no podemos controlar todas las posibles estructuras del documento, y por tanto realizar un mapeo sobre la base de datos será prácticamente imposible.
- **Almacenar el documento en una base de datos nativa XML.** El documento, tanto si es centrado en datos como en texto, se almacena y recupera de forma intacta. Como veremos, será la mejor opción, sobre todo si el documento está basado en texto.

La siguiente imagen ampliable ilustra las anteriores opciones de almacenamiento:



En la actualidad, cada vez más, las BD convencionales ofrecen posibilidades de almacenamiento XML, por lo que se habla de BD XML-compatible o BD XML-enabled. Por tanto, podemos hablar de dos **tipos de Sistemas de Bases de Datos que soportan documentos XML:**

- **BD XML-compatibles**: desglosan un documento XML en su correspondiente modelo relacional o de objetos.
- **BD XML Nativas**: respetan la estructura del documento, permiten hacer consultas sobre dicha estructura y recuperan el documento tal y como fue insertado originalmente

Las principales diferencias entre ambos tipos de BD XML son las siguientes:

- Una BD XML nativa proporciona un modelo de datos propio, mientras que un BD XML-compatible tiene ya un modelo de datos y añade una capa de software que permite de alguna manera almacenar documentos XML y recuperar los datos generando nuevos documentos XML.
- Una BD XML nativa debe manejar todos los tipos de documentos posibles, mientras que una BD XML-compatible solo puede manejar y almacenar los documentos que encajan dentro del modelo definido para ellos.

Para saber más

No dejes de visitar la página de Rounald Bourret, uno de los más importantes investigadores y consultores sobre BD XML. Aunque en inglés, en esta página encontrarás mucha información sobre el desarrollo de las bases de datos XML y sus especificaciones.

Página de Rounald Bourret sobre BD XML.

2.- Bases de Datos Nativas XML.

Caso práctico

Esta mañana, **María** ha estado hablando con una amiga del gremio. Su amiga lleva varios años trabajando en proyectos con tecnologías XML y además ha asistido personalmente a algunas conferencias del mismísimo Rounald Bourret, por lo que está más que al tanto de lo último en estas tecnologías.

María le ha pedido consejo sobre algunas de las bases de datos nativas XML, actualmente en el mercado.

Las **Bases de Datos nativas XML NXD** (Native XML Database) aunque existen desde hace años, en la actualidad siguen evolucionando, por lo que existen ciertas diferencias entre los productos de este tipo que podemos encontrar en el mercado. Lo que si está bastante claro, es que **una NXD o BD XML debe cumplir las siguientes propiedades**:

- **Define un modelo lógico de datos XML,** estableciendo los elementos que son significativos, de forma que el documento XML se pueda almacenar y recuperar de manera intacta, esto es, con todos sus componentes. Por tanto, el modelo, como mínimo, debe tener en cuenta: los elementos, atributos, texto, secciones CDATA, y preservar el orden en el documento.
- **El documento XML es la unidad lógica de almacenamiento,** esto es, la unidad mínima de almacenamiento.
- **No tiene ningún modelo de almacenamiento físico subyacente concreto**. Pueden ser construidas sobre bases de datos relacionales, jerárquicas, orientadas a objetos o bien mediante formatos de almacenamiento propietarios.

Lo que realmente cambia en estas BD respecto a las convencionales, es el formato que soportan, ya que, están especializadas en almacenar documentos XML, almacenarlos y recuperarlos con todos sus componentes.

Otras características de las BD Nativas XML son las siguientes:

- **Documentos y colecciones**. Los documentos se pueden agrupar en unidades denominadas colecciones.
- **Indexación**. Permiten la creación de índices para acelerar las consultas realizadas frecuentemente.
- **Identificador único**. A cada documento XML se le asocia un identificador único, por el que será reconocido dentro del repositorio.
- **Consultas**. Soportan uno o más lenguajes de consulta. Entre ellos, uno de los más populares es XQuery (XML Query)
- **Actualizaciones**. Poseen diferentes estrategias para actualizar documentos, entre ellas la más popular es Update XQuery.
- **Validación**. No siempre se realiza validación de documentos, esto es, no necesitan un **DTD** o un XML Schema (W3C) para almacenar documentos, basta con que los documentos sean XML bien formados.
- Soportan transacciones, accesos concurrentes, control de accesos y backup como cualquier otro sistema de bases de datos.

Y ¿en qué situaciones puede resultar imprescindible su uso? Fundamentalmente en las siguientes situaciones**:**

- Existencia de documentos con anidamientos profundos.
- Importancia de preservar la integridad de los documentos.
- Frecuentes búsquedas de contenido.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Una BD XML nativa solo puede almacenar documentos XML válidos.

Verdadero. ○ Falso. ○

Para saber más

Consulta el siguiente enlace para recordar o bien familiarizarte con lo que son los DTDs y los esquemas XML.

Los DTDs y esquemas XML.

2.1.- Estrategias de almacenamiento.

Podemos diferenciar o clasificar las BD XML nativas en función del tipo de almacenamiento que utilicen, que puede ser:

- Almacenamiento basado en texto
- Almacenamiento basado en el modelo
- Soluciones desarrolladas específicamente para la gestión de documentos XML

¿En qué consiste el almacenamiento basado en texto y basado en modelo?

- El **almacenamiento basado en texto** consiste en almacenar el documento XML entero en forma de texto (fichero de texto), y proporcionar alguna funcionalidad de base de datos para acceder a él.
  Se suelen aplicar técnicas de compresión para reducir el espacio de almacenamiento, utilizar índices adicionales para mejorar el acceso a la información, y se pueden definir sobre BD tradicionales o sistemas de ficheros. Básicamente existen dos posibilidades:
  - Almacenar el documento como un binario largo (BLOB) en una base de datos relacional, o mediante un fichero, y proporcionar algunos índices sobre el documento que aceleren el acceso a la información.
  - Almacenar el documento en un almacén adecuado con índices, soporte para transacciones, etc.
- El **almacenamiento basado en modelo** consiste en definir un modelo de datos lógico, como DOM, para la estructura jerárquica de los documentos XML y almacenar el modelo binario del documento en un almacén existente o bien específico. En esta caso las posibilidades que existen son:
  - Traducir el DOM a tablas relacionales como elementos, atributos, entidades, etc.
  - Traducir el DOM a objetos en una BDOO.
  - Utilizar un almacén creado especialmente para esta finalidad

A continuación, te indicamos algunos ejemplos de BD XML nativas clasificadas según su sistema de almacenamiento:

- Sistema propietario: XIndice, Virtuoso, Tamino XML Server.
- Sistema relacional: eXist, DBCOM, XDB
- Sistema orientado a objetos: Ozone, MindSuite XDB.

¿Y qué ventajas proporcionan las BD XML nativas sobre otros sistemas de almacenamiento? Las **principales ventajas** son las siguientes:

- No necesitan mapeos adicionales.
- Conservan la integridad de los documentos.
- Permiten almacenar documentos heterogéneos.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

El almacenamiento basado en modelo supone almacenar el documento como texto en un almacén adecuado con índices y que soporte transacciones.

Verdadero. ○ Falso. ○

Para saber más

Para refrescar tus conocimientos sobre el modelo de objetos de documentos DOM te recomendamos que visites el siguiente enlace. También obtendrás información sobre el parser SAX (Simple API for

XML):

Los modelos DOM.

2.2.- Colecciones y documentos.

En general, una BD XML tiene una estructura jerárquica organizada en colecciones y documentos XML. La estructura jerárquica comienza con un **nodo raíz ('/')**, del que parten colecciones y documentos.

Una colección:

- Es un conjunto de documentos agrupados, normalmente, en función de la información que contienen.
- Puede contener otras colecciones.

Un documento:

- Información XML
- Información de otro tipo y entonces se le denomina non-XML data (Datos no-XML)

Comparando con una base de datos relacional o un sistema de archivos:

- Las colecciones juegan en las bases de datos nativas el papel de las tablas en las DB relacionales, o de un directorio en un sistemas de archivos.
- Los documentos juegan el papel de las filas de una tabla de una BD relacional o un fichero en un sistema de archivos.

Aunque depende de cada implementación, en muchos casos:

- Cada colección puede tener más de un DOCTYPE asociado.
- El elemento raíz del documento XML define a que DOCTYPE estará asociado el documento, en caso de no poseer ninguno, éste se crea dinámicamente. Esto posibilita el almacenamiento de documentos sin formato definido.
- La colección también puede tener asociado un Schema con información tanto física como lógica de la colección. La parte lógica define las relaciones y propiedades de los documentos XML y la física contiene información sobre el almacenamiento e indexación de los mismos.
- También se pueden almacenar documentos no-XML, para estos existe un DOCTYPE especial llamado non-XML.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Una colección solo puede almacenar documentos XML.

Verdadero. ○ Falso. ○

Citas para pensar

A veces sentimos que lo que hacemos es tan solo una gota en el mar, pero el mar sería menos si le faltara una gota.

Madre Teresa de Calcuta

2.3.- Gestores nativos XML comerciales y libres

Te indicamos a continuación algunos ejemplos de gestores nativos XML clasificados según sean de código libre o código propietario. Los comerciales, generalmente ofrecen versiones de prueba con limitaciones de tiempo de uso y de funcionalidades. Los libres, a veces también ofrecen dualidad de licencia, comercial y libre.

Algunos de estos gestores nativos XML son los siguientes:

- **Gestores nativos XML comerciales** o de código propietario son:
- **TaminoXML Server**. Es un gestor nativo XML de la empresa SoftwareAG. Es un producto comercial de alto rendimiento y disponibilidad, además de ser uno de los primeros SGBD XML nativos disponibles. Algunas de sus características son las siguientes:
  - Los documentos se almacenan en una base de datos propia y no se transforman en otro modelo.
  - Existe un espacio separado para documentos y para índices.
  - Soporta el lenguaje de consultas XQuery y APIs para Java, C, y .NET, entre otras.
- **TEXTML** de Isiasoft. Los documentos se almacenan en su formato nativo, sin ser mapeados.
  - Permite almacenar documentos sin DTD o esquema.
  - Proporciona índices de acuerdo a la propia estructura del documentos.
  - Permite la utilización de varios índices al mismo tiempo.
  - Incluye API para Java, WebDAV, OLE DB y .NET.

- **Gestores nativos XML libres** o de código abierto son:
- **Qizx.** Es una base de datos nativa XML escrita en Java. Almacena los datos utilizando una representación que se basa en el modelo de **XPath** 2,0.
  - La representación utiliza compresión de manera que un documento y sus índices suelen utilizar menos espacio que el original XML.
  - Los documentos se almacenan en una jerarquía de colecciones.
  - Permite almacenar documentos sin DTD o esquema.
  - Soporta el lenguaje de consulta XQuery y sus extensiones, como XUpdate, así como API para Java.
- **eXist**. Utiliza un sistema de almacenamiento propio (árboles B + y archivos paginados). Se puede ejecutar como un servidor de base de datos independiente, como una biblioteca de Java embebida, o en el motor `servlet` de una aplicación Web.
  - Los documentos se almacenan en una jerarquía de colecciones.
  - Permite almacenar documentos sin DTD o esquema.
  - Soporta el lenguaje de consulta XQuery y sus extensiones, como XUpdate, así como API para Java.

Para saber más

Puedes consultar una relación extensa de bases de datos nativas XML, comerciales y libres, en el siguiente enlace:

Relación de BD XML nativas, comerciales y libres.

Si no sabes lo que es un árbol B+, consulta este enlace para conocer estas estructuras de datos:

Información sobre árboles B+

Citas para pensar

El éxito no se logra sólo con cualidades especiales. Es sobre todo un trabajo de constancia, de método y de organización.

J.P. Sergent

3.- Instalación del gestor Qizx.

Caso práctico

Juan y **María** están muy satisfechos con **Ana**, realmente les está ayudando bastante con sus conocimientos de XML. Además, justo el gestor XML nativo que propuso **Ana** desde el principio, es uno de los que a **María** le sugirió su amiga. **Juan** le pregunta a **Ana** —¿Has trabajado antes con el gestor Qizx? **Ana** le contesta —Pues....., sí. En clase, hicimos algunas prácticas con esta BD XML y realmente me gustó, es sencilla y está optimizada para consultas. Además tiene una versión libre. A lo que **Juan** le dice —Pues ahora es tu momento, serás nuestra profesora estos primeros días, para familiarizarnos con Qizx.

En los siguientes apartados trabajaremos con la base de datos nativa XML Qizx, especialmente diseñada para integrarse en una aplicación Java independiente, esto es, para trabajar en modo embebido (embedded). Como tal, se utiliza principalmente como una biblioteca de clases, lo que supone que para utilizarla en una aplicación habrá que añadir los correspondientes archivos `.jar`. También se puede utilizar en un entorno JEE, Servlets o EJB.

Algunas características de Qizx, que iremos viendo en los siguientes apartados, son las siguientes:

- **Consultas e indexación optimizadas**. A diferencia de otras bases de datos nativas XML, que están optimizadas para la actualización masiva de documentos XML, Qizx optimiza las búsquedas y consultas de documentos mediante un sistema de indexación muy eficiente. Sin embargo, la actualización de documentos de gran tamaño no es tan eficiente.
- **Compatible con estándares**. Es totalmente compatible con el lenguaje XQuery y sus extensiones Full-Text XQuery y Update XQuery.
- **No necesita DTD o Esquemas predefinidos**, siempre y cuando el documento XML esté bien formado.
- **Indexado Automático.** Los documentos son indexados automáticamente, aunque permite también personalizar los índices.

Qizx se distribuye en tres ediciones, la profesional, la edición libre (Free Edition), y la Open Source. Trabajaremos con la edición libre que posee todas las características de la versión comercial, con la limitación de que el tamaño máximo de la BD que puede manejar es de 1 GB, suficiente para nuestros ejemplos.

En el siguiente recurso didáctico, encontrarás un tutorial con los pasos a seguir para la descarga del software Qizx y realizar su instalación.

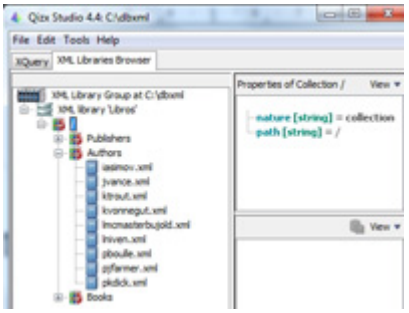
Resumen textual alternativo

Para saber más

En el siguiente enlace, en inglés, encontrarás un extenso y completo tutorial sobre Qizx.

Tutorial oficial de Qizx.

3.1.- Gestionar la base de datos XML mediante QizxStudio.



Una BD XML Qizx está organizada como una estructura jerárquica de colecciones, que a su vez contienen documentos, similar a un sistema de archivos, donde las colecciones juegan el papel de los directorios y los documentos XML desempeñan el papel de los archivos planos. Dentro de una colección pueden almacenarse documentos de cualquier tipo.

Te detallamos a continuación la terminología utilizada por **Qizx**:

- **XML `Library`**. Es el equivalente a una base de datos XML. Físicamente, una BD o `Library` se almacena en un directorio en un disco, y no hay límite en el número de bases de datos que se pueden crear con Qizx..
- Su estructura jerárquica siempre comienza con un nodo raíz, `'/'`, del que parten documentos XML y/o colecciones.
- **XML `Library` Group.** Es un conjunto de BD XML, agrupadas dentro de un directorio en disco.

Por ejemplo, la imagen ampliable de la derecha muestra:

- **XML `Library Group at C:\dxml`**. Es el grupo de BD XML situado en el directorio físico `c:\dbxml`.
- **XML `Library` 'Libros'.** Es la BD XML de nombre `Libros`.
- `'/'`. Es el nodo raíz de la BD o Library
- **Las colecciones** `Publisher`, `Authors` y `Books` dentro de la BD XML de nombre `Libros`.
- **Diferentes documentos** (archivos XML) dentro de la colección `Authors`, como por ejemplo `iasimov.xml`.

Podemos trabajar directamente con Qizx a través de la herramienta gráfica denominada **QizxStudio,** la cual **nos permite entre otras acciones las siguientes:**

- Crear la BD XML
- Explorar colecciones y documentos XML de la base de datos
- Importar, crear y eliminar colecciones
- Importar y eliminar documentos, así como copiar o exportar a fichero
- Consultar documentos

Antes de comenzar a trabajar con Qizx y Java, y con el objetivo de que te familiarices con esta BD XML, verás algunos ejemplos de cómo gestionar colecciones y documentos mediante QizxStudio.

En el siguiente recurso didáctico encontrarás ejemplos de todo ello.

Resumen textual alternativo

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un XML Group Library es el equivalente a una BD XML.

Verdadero. ○ Falso. ○

4.- El lenguaje de consultas XQuery.

Caso práctico

Ana está orgullosa en su nueva faceta de profesora. **Juan** y **María** la han felicitado y están satisfechos con la introducción que les ha dado sobre Qizx. **Ana** les dice —Antes de empezar con la aplicación en Java, necesitamos familiarizarnos con el lenguaje de consultas XQuery.

A lo que **Juan** le pregunta —¿Conoces algo del lenguaje?"

Ana sonríe y contesta —Pues...sí. En clase vimos también algunos ejemplos sencillos de uso.

Juan, con cara de asombro dice —Está claro que el nuevo Ciclo Formativo de DAM incorpora muchos temas punteros que yo no domino. El próximo curso iniciaré sin falta el estudio de algunos módulos.

XQuery es un lenguaje de consulta diseñado para extraer información de colecciones de datos expresadas en XML**.** Entre las **principales características de XQuery** vamos a destacar las siguientes:

- Está **basado en el lenguaje XPath, (**XML Path Language), y se fundamenta en él para realizar la selección de información y la iteración a través del conjunto de datos XML.
- Es un **lenguaje declarativo**, lo que significa que, en vez de ejecutar una lista de comandos como un lenguaje procedimental clásico, cada consulta es una expresión que es evaluada y devuelve un resultado, al igual que en SQL.

Podemos decir que **XQuery es a XML lo mismo que SQL es a las bases de datos relacionales.** Sin embargo, aunque XQuery y SQL puedan considerarse similares, el modelo de datos sobre el que se sustenta XQuery es muy distinto del modelo de datos relacional sobre el que sustenta SQL, ya que XML incluye conceptos como jerarquía y orden de los datos que no están presentes en el modelo relacional.

Por ejemplo, a diferencia de SQL, en XQuery es importante y determinante el orden en que se encuentren los datos, ya que no es lo mismo buscar una etiqueta **`nombre`** dentro de una etiqueta **`autor`** que todas las etiquetas **`nombre`** del documento (que pueden estar anidadas dentro de una etiqueta **`autor`** o fuera).

Los **principales tipos de expresiones de XQuery** son:

- Expresiones XPath, para navegar por los documentos.
- Expresiones `FLWOR (For, Let, Where, Order, Return)` para iterar por los elementos de un conjunto de datos.

Pero XQuery también admite:

- Constructores para generar nodos y contenido dinámico.
- Condicionales (`IF, THEN ELSE`) para construir el resultado en base a alguna condición.
- Cuantificadores (`SOME, ANY`) para chequear la existencia de algún elemento que cumpla una condición.
- Listas a las que se pueden aplicar operadores (`UNION`,...) y funciones de agregación (`AVG`, `COUNT`,...).

Para saber más

En este enlace puedes profundizar sobre el lenguaje XQuery.

Especificación completa de XQuery.

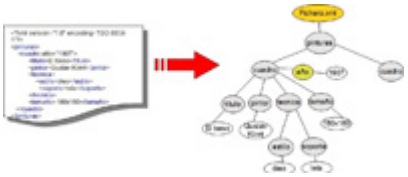
El siguiente enlace te propociona a su vez enlaces muchos enlaces interesantes a varias tecnologías asociadas a XQuery, así como a XQuery.

Tecnologías asociadas con XQuery.

4.1.- Modelo de datos.

El modelo de datos en que se sustenta XQuery es el modelo de datos de XPath.

XPath modela un documento XML como una estructura jerárquica en forma de árbol. El árbol está **formado por** nodos**, y hay siete tipos de nodos**: raíz, elemento, texto, atributo, espacio de nombres, instrucción de procesamiento y comentario.



Los **principales nodos** de la estructura jerárquica o en árbol en un documento XML son: (puedes verlos en la imagen ampliable superior)

- **Nodo raíz o** /. Es el primer nodo del documento.
- **Nodo elemento**. Cualquier elemento de un documento XML. Cada nodo elemento posee un padre y puede o no tener hijos. En el caso de que no tenga hijos, es un nodo hoja.
- **Nodo texto**. Cualquier elemento del documento que no esté marcado con una etiqueta de la DTD del documento XML.
- **Nodo atributo**. Un nodo elemento puede tener etiquetas que complementen la información de ese elemento. Esto sería un nodo atributo.

Debes conocer

En el siguiente enlace tienes una ilustración gráfica más detallada del paso de un documento XML a un árbol de nodos XML, así como la explicación en detalle de cada tipo de nodo. (Apartado 2.Modelo de datos XPath)

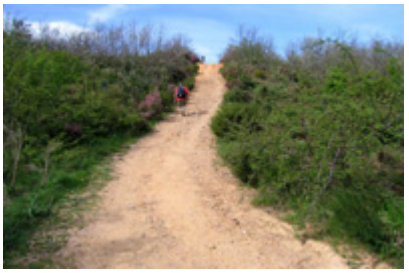
Nodos de un árbol XML.

Autoevaluación

Señala la opción correcta. Entre los nodos de la estructura jerárquica de un árbol XML están:

- Nodo etiqueta y nodo atributo.
- Nodo padre.
- Nodo hijo.
- Nodo raíz y nodo elemento.

4.2.- Caminos de localización.



¿Cómo se localiza cada uno de esos nodos en al árbol XML? Mediante una expresión XPath conocida como camino o ruta de localización**. Un camino de localización:**

- Selecciona un conjunto de nodos relativo al nodo de contexto.
- Puede contener recursivamente expresiones utilizadas para filtrar conjuntos de nodos.
- Al ser evaluado, devuelve el conjunto de nodos seleccionados por el camino de localización.
- Se construye siguiendo unas reglas de sintaxis y semántica.

Hay dos **tipos de caminos de localización:**

- **Caminos relativos.** Son una secuencia de uno o más pasos de localización separados por **/.**
  - Los pasos se componen de izquierda a derecha.
- **Caminos absolutos.** Consiste en / seguido, opcionalmente, por un camino de localización relativo.
  - Una / por si misma selecciona el nodo raíz del documento que contiene al nodo contextual.

Los siguientes, son **algunos ejemplos de caminos de localización:**

- **`cuadro`** selecciona los elementos `cuadro` hijos del nodo contextual.
- **`cuadro//titulo`** selecciona los elementos `titulo` descendientes de los elementos `cuadro` hijos del nodo contextual.
- ***** selecciona todos los elementos hijos del nodo contextual.
- **`@año`** selecciona el atributo `año` del nodo contextual
- **`@*`** selecciona todos los atributos del nodo contextual
- **`cuadro[1]`** selecciona el primer hijo `cuadro` del nodo contextual
- **`cuadro[@año=1907]`** selecciona todos los hijos `cuadro` del nodo contextual que tengan un atributo `año` con valor `1907`.

Debes conocer

En el siguiente enlace tienes ejemplos de caminos de localización en la sintaxis abreviada de XPath (Es el apartado 2.5 Sintaxis abreviada)

Caminos de localización XPath.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

El camino: //curso selecciona los elementos curso descendientes del nodo contextual.

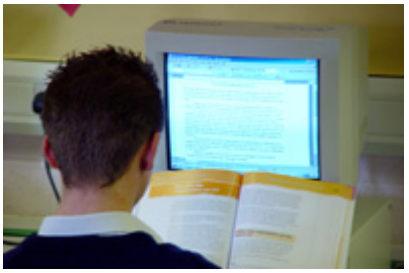
Verdadero. ○ Falso. ○

Citas para pensar

Razonar y convencer, ¡qué difícil, largo y trabajoso! ¿Sugestionar? ¡Qué fácil, rápido y barato!

Santiago Ramón y Cajal

4.3.- Primeras consultas XQuery.



Una **consulta XQuery es una expresión que lee una secuencia de datos en XML y devuelve como resultado otra secuencia de datos en XML**, donde:

- Una **secuencia** es un conjunto ordenado de cero o más ítems.
- Un **ítem** es cualquier tipo de nodo del árbol XML o un valor atómico.

Las funciones que se pueden invocar para **referirnos a colecciones y documentos dentro de la BD** son las siguientes:

- `collection(camino de la colección)`
- `doc(camino del documento)`

Así por ejemplo:

- La consulta `collection(/Books)`: devuelve el contenido de la colección de ruta absoluta `/Books`.
- La consulta `doc(/Empresa.xml)`: devuelve el documento `/Empresa.xml` completo.

Otros **ejemplos de consultas** XQuery **basadas en expresiones** XPath son los siguientes:

- La consulta `collection(/Books)//book/title` devuelve los nodos `title` de todos los libros (`book`) de la colección `/Books`
- Si se utilizan espacios de nombres o `namespaces`, entonces la consulta anterior se redactaría de la siguiente forma: `declare namespace t = http://www.qizx.com/namespace/Tutorial; collection(/Books)//t:book/t:title`
- La consulta `doc(/Empresa.xml)//nombre` devuelve todos los nodos `nombre` del documento `/Empresa.xml`

En la siguiente presentación hay diferentes ejemplos de consultas XQuery ejecutadas directamente sobre una sencilla BD XML denominada '**`Cursillos`**'. Necesitarás descargar el archivo:

Acceso a los ejemplos. (0.01 MB)
Resumen textual alternativo

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

La expresión `colletcion("/Cursos")//curso[aula=2]/profesor` devuelve los cursos y los profesores con cursos en el aula 2.

Verdadero. ○ Falso. ○

Para saber más

Te recomendamos que visites el siguiente enlace para profundizar en el concepto y uso de los **`namespaces`**.

Espacios de nombres o **`namespaces`** en XML.

4.4.- Expresiones FLWOR.

Los ejemplos de consultas XQuery vistos en el apartado anterior, son la manera más sencilla de realizar búsquedas y selecciones de nodos concretos en una BD XML. Pero existe otra manera mucho más potente de realizar este trabajo, mediante lo que se denomina consultas o **expresiones FLWOR** (leído como flower), siendo FLWOR las siglas de For, Let, Where, Order y Return.

Se trata de una **expresión que permite la unión de variables sobre conjuntos de nodos y la iteración sobre el resultado**. Las diferentes **cláusulas de una expresión FLWOR** son:

- **`For`**. Permite seleccionar los nodos que se quieren consultar, guardándose su valor en una variable (identificador que comienza por $). Al conjunto de valores de la variable se le llama **tupla**.
- **`Let`**. (opcional). Asocia valores a variables.
- **`Where`** (opcional). Permite filtrar los resultados según una condición.
- **`Order`** (opcional). Permite ordenar la secuencia de valores o resultados.
- **`Return`**. Genera los valores de salida o devueltos.

En la siguiente imagen, puedes ver un ejemplo de una consulta (izquierda) donde aparecen las 5 cláusulas. La consulta devuelve los nombres de los cursos con 20 plazas, ordenados por nombre de curso (derecha).

Una expresión FLWOR vincula variables a valores con las cláusulas for y let y utiliza esos vínculos para crear nuevas estructuras de datos XML.

A continuación se muestra otro ejemplo de consulta XQuery. La siguiente consulta (izquierda) devuelve los nombres de los cursos con cuota mensual (derecha). Como **`cuota`** es un atributo del elemento **`precio`**, recuerda que se le antecede con un carácter **`@`**.

En el siguiente recurso didáctico dispones de más ejemplos de consultas XQuery, utilizando expresiones FLOWR.

Resumen textual alternativo

Para saber más

En el siguiente enlace tienes muchos ejemplos de consultas XQuery con expresiones FLWOR.

Ejemplos XQuery con expresiones FLWOR

4.5.- XQuery Update Facility.

El lenguaje **XQuery solo proporciona expresiones para la realización de consultas sobre documentos XML**, pero **no su actualización (inserción, modificación o eliminación de nodos)**.

XQuery Update Facility es una extensión de XQuery que permite la actualización de documentos mediante las cláusulas **`insert, delete, replace y rename`**.

El **funcionamiento de las cláusulas de XQuery Update** es el siguiente:

- **`insert`**. Permite la inserción de uno o varios nodos antes (`before`) o después (`after`) del nodo indicado. También se puede insertar al principio (`as first into`) o al final del documento (`as last into`).
- **`delete`.** Elimina uno o varios nodos del documento.
- **`replace`**. Tiene dos funciones:
  - Modificar el valor del nodo
  - Modificar el nodo completo
- **`rename`.** Renombra un nodo (elemento, atributo o instrucciones de proceso) sin afectar a su contenido.

A continuación te mostramos dos ejemplos sencillos, uno de **`insert`** y otro de **`delete`**:

- Eliminar la empresa de `id=2` en el documento Empresa.xml: `delete node doc(Empresa.xml)//empresa[@id=2]`
- Insertar el nodo tipoAccesible/tipo al final del documento `/Aulas/aula3 insert node tipoAccesible/tipo as last into doc(Aulas/aula3.xml)//aula`

Resumen textual alternativo

En el siguiente enlace puedes otros ejemplos de actualizaciones usando XQuery Update:

Ejemplos de XQuery Update

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Mediante rename se puede sustituir el valor de un nodo.

Verdadero. ○ Falso. ○

Para saber más

Desde este enlace puedes acceder al tutorial de Qizx sobre XQuery Update:

Tutorial de Qizx sobre XQuery Update.

En el siguiente enlace puedes ver un documento tipo resumen con muchos de los aspectos tratados en esta unidad hasta el momento.

XML y Bases de Datos.

5.- Trabajar con colecciones y documentos desde Java.

Caso práctico

—¡Ha llegado la hora de manejar la base de datos desde Java! —exclama **Ana**, y añade— ¿queréis que os indique cómo crear colecciones y añadir documentos?

Juan contesta —Pues claro **Ana**, ¿sabes cual es el API principal que proporciona este gestor para trabajar con Java?

Ana contesta —De memoria no lo recuerdo, pero sí recuerdo que la propia distribución dispone de datos de prueba que podemos utilizar en nuestros primeros ejemplos.

María, que escucha con atención dice —¡Perfecto!, vamos a ello.

El **API principal para trabajar con una base de datos XML de Qizx** en un entorno de desarrollo Java, la proporciona **el paquete com.qizx.api**, que permite, entre otras cosas, realizar las siguientes tareas:

com.qizx.api
ManagerLibrary Library

- crear la BD XML,
- abrir y cerrar conexiones a la BD,
- importar/crear colecciones y documentos, y
- realizar consultas con XQuery.

A continuación te indicamos algunas de las **clases e interfaces del paquete `com.qizx.api`** de más interés:

- Interface `Library`. Representa una BD XML o una conexión a una BD XML.
- Interface `LibraryManager`. Representa el gestor de las BDs. Permite gestionar grupos de BDs XML.
- Clase `Configuration`. Inicializa y configura un `LibraryManager`, permitiendo su creación, apertura, etc..
- Interface `LibraryMember`. Representa un miembro de una BD XML, esto es, colección o documento.
  - Algunas sub_interfaces son: `Collection, Document, NonXMLDocument`.
- Interface `Collection`. Para gestionar colecciones y sub_colecciones que contienen documentos XML.
- Interface `Document`. Representa documentos XML bien formados y almacenados en una BD XML.
  - Al ser un LibraryMember (miembro de una Library), puede tener propiedades (metadatos) que pueden consultarse.
- Clase `Index`. Para crear índices en una BD XML y especificar sus parámetros.

Otros paquetes de interés son:

- El **paquete `org.xml.sax`**. Proporciona las clases e interfaces para el API SAX, que es un componente de la API de Java para el procesamiento de documentos XML, como por ejemplo el interface `XMLReader`.
- El **paquete `com.qizx.xdm`**. Proporciona clases e interfaces para el análisis y procesamiento de documentos XML, como por ejemplo la clase `DocumentParser`.

En el siguiente recurso didáctico encontrarás un tutorial con los pasos a seguir para la integración de Qizx en el IDE NetBeans, y poder hacer uso de las clases e interfaces comentadas anteriormente.

Resumen textual alternativo

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

El interface Document , representa documentos XML bien formados y almacenados en una

base de datos XML.

Verdadero. ○ Falso. ○

Para saber más

En el siguiente enlace puedes consultar el API de Qizx en toda su extensión.

Documentación oficial del API Qizx.

5.1.- Creación de la BD XML. Establecer conexiones.

En general, la conexión de una aplicación Java a una base de datos XML se realizará mediante el API proporcionado por el propio gestor XML.

Para crear una BD XML y establecer una conexión se utilizan **métodos del interface LibraryManager.**

- **Crear la BD XML. El método `createLibrary(String nombreBD, File directorioBD)` crea la BD de nombre `nombreBD` localizada físicamente en el directorio `directorioBD`, directorio que debe existir y estar vacío. Si el parámetro `directorioBD` se pasa como valor `null`, la BD se creará en un subdirectorio del grupo con nombre `nombreBD`.**
- **Establecer conexión**. El método `openLibrary(String nombreBD)` abre una nueva sesión o conexión a la BD de nombre `nombreBD`. También es posible, indicar parámetros de autentificación de usuario, si se ha establecido un Control de Accesos a la `Library` o BD.

En el siguiente ejemplo puedes ver un método que realiza la conexión o apertura a una base de datos XML, y si no existe la crea y después la abre.

Código Java para la conexión o apertura una biblioteca XML (1 KB)

Si observas el ejemplo anterior (valor devuelto por el método), verás que en Qizx, un objeto **`Library`** representa dos cosas:

- La base de datos XML
- Una conexión concreta a la base de datos o `Library` XML

Debes conocer

En el siguiente enlace encontrarás una explicación más detallada de la dualidad del interface **`Library`**: como Base de datos y como conexión.

El objeto Library como BD y como conexión.

Para saber más

En el siguiente enlace puedes consultar cómo establecer un control de accesos a una BD XML.

Control de Acceso en Qizx.

5.2.- Cerrar conexiones.

Podemos cerrar la conexión de una BD o **`Library`** concreta, mediante el método **`close()`** de la interface **`Library`**, o cerrar todas las conexiones a las bases de datos de un grupo, mediante el método **`closeAllLibraries()`** de la interface **`LibraryManager`**.

Te detallamos cada caso:

- **Cerrar una conexión**. El método `Library.close()` cierra la conexión a una BD particular. Si ha comenzado alguna transacción, no se podrá invocar `close()`, solo se podrá invocar `commit()` o `rollback()`, por lo que antes de invocar `close()`, debemos comprobar mediante el método `Library.isModified()` si la base de datos está inmersa en una transacción.
- **Cerrar todas las conexiones**. El método `LibraryManager.closeAllLibraries(int tiempo)` cierra todas las conexiones a BD del mismo grupo y libera los recursos. Mediante el parámetro `int tiempo`, se le puede indicar el tiempo en milisegundos que debe esperar el sistema antes de realizar la desconexión de aquellas `Library` o bases de datos en las que haya alguna transacción sin finalizar. Pasado ese tiempo, se generará de forma automática un `rollback()`, se deshacen las operaciones, en aquellas transacciones que no hayan finalizado aún.

En el siguiente ejemplo, puedes ver cómo realizar la desconexión o cierre tanto de una BD particular, como de todo un grupo de bases de datos o **`Library`**.

```
//método que cierra la conexión a la base de datos
private static void cerrar(Library bd, LibraryManager bdManager)
        throws QizxException {
  //Si la base de datos está inmersa en una transacción
  if (bd.isModified()) {
    //Deshace los cambios realizados por la transacción
    bd.rollback();
  }
  //cierra la conexión a la base de datos bd
  bd.close();
  //cierra las bases de datos del grupo después de 10000 ms
  bdManager.closeAllLibraries(10000 /*ms*/);
}
```

Código Java para la desconexión o cierre de una biblioteca XML. (1 KB)

Citas para pensar

"El sabio puede sentarse en un hormiguero, pero sólo el necio se queda sentado en él."

Proverbio chino.

Autoevaluación

Señala la opción correcta. Para cerrar una conexión a una base de datos Qizx se utiliza el método:

- Library. closeAllLibraries().
- LibraryManager.close().
- Library.disconnect().
- close() del interface Library

5.3.- Crear colecciones.

Para **crear colecciones** se usan los métodos:

- `createCollection(String ruta)`. Método de la interface `Library`. Crea una colección en la BD, cuya posición dentro de la base de datos es la que indica el parámetro `ruta`, siendo `ruta` una camino o path absoluto en la BD.
- `createChildCollection(String nombre)`. Método de la interface `Collection`. Crea una sub_colección dentro de una colección, denominada `nombre`.

Por ejemplo, en el siguiente segmento de código se estaría creando dentro de la **`Library`** o base de datos **`bd`** una colección situada dentro de la base de datos en la posición indicada por **`ruta`**. Observa que, previamente se comprueba si ya existe esa colección dentro de la base de datos.

```
Collection coleccion = bd.getCollection(ruta);
//Si no existe la coleccion, crea la coleccion
if (coleccion == null) {
  System.out.println("Creando coleccion '" + ruta + "'...");
  coleccion = bd.createCollection(ruta);
}
```

Código Java para la creación de una colección en una BD XML. (0.01 MB)

Citas para pensar

"Saber que se sabe lo que se sabe y que no se sabe lo que no se sabe; he aquí el verdadero saber."

Confucio

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Para crear una colección en una BD XML se utiliza el método createChildCollection() de la interface Collection.

Verdadero. ○ Falso. ○

5.4.- Añadir documentos.

Para **añadir documentos** se utiliza el método:

- `importDocument(String documentoRuta, Tipo fuente)`.Método de la interface `Library`. Existen diferentes métodos para importar documentos XML a una BD XML, que se diferencian fundamentalmente en el `Tipo` de los documentos `fuente` indicados como segundo parámetro: `java.io.File`, `java.net.URL`, `org.xml.sax.InputSource`, etc. En todos los casos, la fuente debe contener un documento XML bien formado. Uno de estos métodos es:
  - `importDocument(String documentoRuta, File fichero)`. Importa un documento a la base de datos mediante un análisis o parsing de un `fichero` de texto XML ubicado en fichero.
    - Si ya existe un documento con la misma ruta en la BD, se sustituye por los nuevos contenidos.
    - Si la colección que contiene este documento no existe, se crea automáticamente.

El siguiente código Java, es un ejemplo de cómo importar documentos mediante el método **`importDocument()`**:

```
System.out.println("Importando '" + fichero + "' como documento '" + ruta
        + "'...");
//importa a BD, en la posición indicada por ruta, el documento XML
// cuyo texto XML está en fichero
BD.importDocument(ruta, fichero);
```

Código Java para la importación de un documento a una BD XML. (0.01 MB)

Qizx necesita analizar un documento XML con el fin de poder importarlo a la base de datos. El primer paso del análisis consiste en comprobar que se trata de un documento bien formado. También se puede realizar su validación, mediante la comprobación del DTD. Si falla este paso, fallará la importación.

Con Qizx, se puede utilizar lo que se conoce como **catálogo XML** para realizar este análisis o comprobaciones, sin necesidad de acceder a direcciones externas u otras localizaciones que puedan suponer un fallo si no las encuentra en el sistema local.

Qizx incluye la biblioteca resolver.jar, conocida como **resolución del catálogo XML**, que forma parte del proyecto Apache XML Commons, de manera que todos los métodos **`Library.importDocument()`** son catálogos XML habilitados. Por lo tanto, basta con especificar el correspondiente **`importDocument()`** y dejar que mediante **`resolver.jar`** se realice la resolución, análisis o parsing.

Debes conocer

Consulta el siguiente enlace para conocer los diferentes métodos importDocument() que puedes utilizar.

Diferentes métodos **`importDocument()`**.

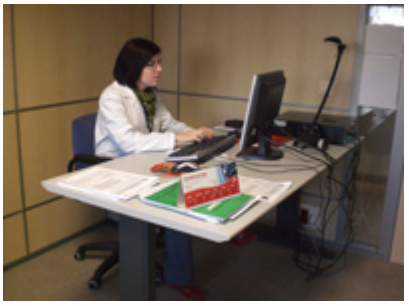
Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

La única forma de importar documentos a na BD XML, es mediante el método Library.importDocument(String documentoRuta, File fichero).

Verdadero. ○ Falso. ○

5.5.- Creación de una BD XML e importación de colecciones y documentos.



Otros métodos útiles al llenar una **`Library`** con colecciones y documentos, o bien trasladar colecciones y documentos de una **`Library`** a otra son los siguientes:

- `Library.getMember(String rutaAbsoluta)`: devuelve el miembro especficado en esa ruta de la BD, un `LibraryMember` (documento o colección), o bien `null` si no existe ese miembro.
- `Library.getDocument(String rutaAbsoluta)`: devuelve el documento especficado en esa ruta de la BD, un `Document` o bien `null` si no existe.
- `Library.getCollection(String rutaAbsoluta)`: devuelve colección especficada en esa ruta de la BD, una `Collection` o bien `null` si no existe.
- `Library.getName()`: devuelve el nombre de la `Library` o base de datos.
- `LibraryMember.getPath()`: devuelve la ruta completa del miembro en esa BD.
- `LibraryMember.getName()`: devuelve el nombre del miembro dentro de su colección sin `'/'`, excepto si es la colección raríz.
- `LibraryMember.isDocument()`: devuelve verdadero si el miembro es un documento.
- `LibraryMember.isCollection()`: devuelve verdadero si el miembro es una colección.
- `Collection.getChildren()`: devuelve un iterador, un `LibraryMemberIterator`, sobre los miembros de esa colección (documentos o colecciones).
- `LibraryMemberIterator.iter.moveToNextMember()`: devuelve verdadero si accede al siguiente miembro.
- `LibraryMemberIterator.getCurrentMember()`: devuleve el mimbro actual.

En este apartado, verás el ejemplo completo de cómo crear una base de datos XML desde un entorno Java, y llenarla con colecciones y documentos existentes, esto es, importando colecciones y documentos a la base de datos.

Recuerda tener desconectado el **`Library Group`** en QizxStudio cuando ejecutes tus programas Java.

En el siguiente recurso didáctico se explica cómo crear desde Java una BD XML y llenarla con colecciones y documentos.

Resumen textual alternativo

Proyecto Java para la creación de una BD XML con colecciones y documentos . (0.02 MB)

Citas para pensar

" No basta saber, se debe también aplicar. No es suficiente querer, se debe también hacer."

Johann Wolfgang Goethe

5.6.- Eliminar colecciones y documentos.

Para eliminar colecciones y documentos se pueden utilizar los métodos:

- `deleteMember(String ruta)`. Método de la interface `Library`. Elimina el miembro de la BD indicado mediante el parámetro `ruta`, que será la ruta absoluta dentro de la BD. Si se trata de una colección, elimina también de forma recursiva todos sus documentos y sub_colecciones.
- `delete()`. Método de la interface `LibraryMember`. Es equivalente al método anterior, es decir, es equivalente a `Library.deleteMember(getPath())`.

Al igual que otras operaciones de actualización , para que tenga efecto permanente, debe ir seguida de una confirmación o **`commit()`**.

Por ejemplo, si **`bdNombre`** vale **`Libros`** (BD creada anteriormente con qizxStudio, el siguiente segmento de código borra cada una de las colecciones de esta base de datos que se hayan indicado en el **`array String[] coleccionNombre = {"/Authors","/Books"}`**

```
//Abre la base de datos
Library bd = bdManager.openLibrary(bdNombre);
try {
  //Borra las colecciones especificadas
  for (int i = 0; i < coleccionNombre.length; i++) {
    ruta = coleccionNombre[i];
    if (bd.deleteMember(ruta)) {
      System.out.println("Borrando colección '" + ruta
          + "' de la BD '" + bdNombre + "'...");
    }else
      System.out.println("Colección '" + ruta + "' de la BD '"
          + bdNombre + "' no encontrada");
  }
  System.out.println("Confirmando los cambios...");
  bd.commit();
} finally {
  //Cierra la base de datos
  cerrar(bd, bdManager);
}
```

Si por ejemplo, **`ruta`** vale **`/Publishers/Pocket.xml`**, lo que estaríamos borrando con la sentencia **`bd.deleteMember(ruta)`**; sería exactamente el documento **`Pocket.xml`**.

¿Y **cómo eliminar toda la base de datos**? Para ello, puedes utilizar:

- el método `deleteLibrary(String bdNnombre)` de la interface `LibraryManager`, que elimina físicamente la BD.

Desde el siguiente enlace te puedes descargar el proyecto java completo.

Proyecto Java para la eliminación de colecciones y documentos. (0.01 MB)

Autoevaluación

Señala las opciones correctas. Para eliminar una colección:

- ☐ Se puede utilizar el método LibraryMember.delete().
- ☐ Se puede utilizar el método Library.deleteMember().
- ☐ No se puede eliminar si tiene colecciones.
- ☐ No se puede eliminar si tiene documentos.

Mostrar Información

5.7.- Consultar documentos

¿Se utiliza el lenguaje XQuery para realizar consultas a la BD XML desde una aplicación Java? La respuesta es sí, pero será necesario realizar los siguientes **pasos para procesar una consulta desde Java**:

- **Compilar la consulta o expresión XQuery** mediante el método `Library.compileExpression()`. Si la compilación se realiza sin errores (`CompilationException`), devolverá un objeto `Expression`.
- **Evaluar la expresión o consulta compilada** mediante el método `Expression.evaluate()`. Si la evaluación se realiza sin errores (`EvaluationException`), devolverá el resultado en forma de un `ItemSequence`.
- **Iterar sobre la secuencia obtenida.** Un `ItemSequence` permite iterar sobre una secuencia de Items, donde un `Item` es un valor atómico o un nodo XML.

Por ejemplo:

- Expresión `3+4` : se evalúa como el valor atómico 7
- Expresión `collection("/Books")//book/title:` se evalúa como una secuencia de nodos `title`

Un iterador siempre tiene un método **`moveToNextItem()`** que mueve la posición del cursor al siguiente elemento y un **`getCurrentItem()`** que devuelve el elemento encontrado en la posición actual del cursor.

Por tanto, los pasos anteriores suponen las siguientes sentencias de código:

```
Expression exprcompilada = bd.compileExpression(scriptConsulta);
ItemSequence resultado = exprcompilada.evaluate();
while (resultado.moveToNextItem()) {
    Item resul = resul.getCurrentItem();
/* Manipular resultado*/
}
```

Otros métodos útiles en el procesamiento de una consulta son los siguientes:

- El método `Item.getNode()` que obtiene el valor de un nodo XML (objeto tipo `Node`) tras la evaluación de una consulta XQuery compilada.
- El método `XMLSerializer.serializeToString()` que permite serializar como una cadena un nodo XML.
- El método `Expression.bindImplicitCollection()` que permite escribir consultas con rutas o paths las cuales no van precedidas por las expresiones `collection(XXX)` o `doc(YYY)`.

Por ejemplo, usando **`bindImplicitCollection()`** se puede utilizar como expresión: **`//book/title`** en vez de **`collection(/Books)//book/title`**

- El método `Expression.bindVariable()` que permite vincular una secuencia de `Items` a una variable.

En el siguiente enlace dispones de un ejemplo detallado de consultas XQuery desde una aplicación Java. Las consultas se realizan sobre la BD Cursillos que creaste anteriormente. Las consultas que se ejecutan están almacenadas como **`scripts`** en ficheros de extensión **`.xq`** dentro del directorio **`C:\BDCursillosXML\cursillos_query\`**. También puedes probar con este mismo programa las consultas **`7.xq`** y **`8.xq`** que vienen con la distribución Qizx Free Engine Edition y que encontrarás en la carpeta de instalación dentro del directorio **`\docs\samples\book_queries\`**. En este caso, la BD a utilizar será Tutorial.

Proyecto Java para consultar documentos . (0.01 MB)

Debes conocer

Es importante que conozcas las posibilidades del interface **`Expression`** y del interface **`ItemSequence`**.

Interface Expression.

Interface ItemSequence.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un ItemSequence es una secuencia de nodos XML.

Verdadero. ○ Falso. ○

5.8.- Actualizar documentos.

Aunque dependiendo del sistema de BD XML pueden existir diferentes formas de actualizar documentos, la manera más sencilla de hacerlo es mediante **Update XQuery, la extensión del lenguaje XQuery que permite inserciones, eliminaciones y modificaciones de nodos de los documentos XML.**

Con Qizx, independientemente del método utilizado para la actualización, ésta siempre implica sustituir el documento prácticamente en su totalidad. Esto es así debido a que de esta forma se consigue optimizar la velocidad de las consultas.

Para **ejecutar una consulta de actualización desde Java** se siguen básicamente los mismos pasos que para una consulta normal XQuery:

- Compilar la consulta de actualización mediante `Library.compileExpression()` obteniendo un objeto `Expression`.
- Evaluar la consulta de actualización compilada mediante `Expression.evaluate()` obteniendo un `ItemSequence`.

En el siguiente enlace dispones de un ejemplo detallado de actualización XQuery desde una aplicación Java. La actualización consiste en añadir dos nodos empresa al documento **`/Empresa.xml`** de la BD Cursillos. La consulta de actualización está almacenada como un script en el fichero **`12update_insert.xq`** dentro del directorio **`C:\BDCursillosXML\cursillos_query\`**.
En ese directorio dispones de otros scripts de actualización para realizar eliminaciones y modificaciones, que puedes probar desde el mismo programa. (**`13update_delete.xq`**, **`14update_replace_node_value.xq`**, **`15update_rename_varios_nodos.xq`**)

Proyecto Java para actualizaciones sencillas de documentos. (0.01 MB)

El siguiente ejemplo actualiza un documentos de la colección **`Authors`**, insertando un nuevo pseudónimo al autor que se pasa como parámetro al **`script`**. El **`script`** de actualización está almacenado dentro del directorio de instalación de Qizx: **`docs\samples\programming\edit\pseudonym.xq.`** La BD utilizada es Tutorial. Para esta actualización se usa la interface **`QName`**, que permite gestionar nombres calificados. Observa el **`script pseudoynm.xq`**, que viene con tu distribución descargada, para comprobar que todas las referencias a **`QName`** lo hacen con Q mayúscula. Además, observa cómo se utiliza una consulta de actualización con sentencias alternativas **`if-else`** para detectar errores en el proceso de actualización.

Proyecto Java para actualización más compleja de documentos. (0.01 MB)

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Un ItemSequence es una secuencia de nodos XML.

Verdadero. ○ Falso. ○

Para saber más

Para recordar mejor lo que son los nombres cualificados o **`QName`** puedes acceder al enlace:

Nombre cualificados.

Otra estrategia en Qizx para actualizar documentos es utilizar el API DOM. En el siguiente enlace tienes una ejemplo de cómo hacerlo.

Ejemplo de actualización de documentos mediante el API DOM.

6.- Indexación.

Caso práctico

Esta mañana, **María**, **Juan** y **Ana** se han reunido con **Ada** para hablar de los avances del proyecto. Ada, ha quedado bastante satisfecha y les ha preguntado —¿Habéis considerado la posibilidad de establecer índices personalizados para optimizar las consultas?

María contesta —¡Claro **Ada**! Si mal no recuerdo, el personal de la academia de inglés nos insistió en que algunas búsquedas o consultas de información resultaban eternas.

Juan añade —Hoy vamos a tratar precisamente este tema.

Como cualquier otra BD XML, Qizx utiliza índices para optimizar y aumentar la velocidad de las consultas realizadas a la BD.

De manera predeterminada, Qizx indexa la mayor parte de la información disponible en documentos XML: elementos, atributos, nodos, y texto completo. Esto se realiza automáticamente, por lo tanto, en la mayoría de los casos no hay necesidad de especificar explícitamente nuevos índices.

Qizx soporta los siguientes **tipos de índices**:

- **Índices elemento**. Dado un nombre de elemento, este índice devuelve todos los elementos XML en todos los documentos de una BD con este nombre.
- **Índices de atributos**. Dado un nombre de atributo y un valor, este índice devuelve todos los elementos que tienen un atributo con este nombre y valor. Hay **tres tipos** de índices de atributos, de acuerdo con el tipo del valor del atributo: texto, numéricos y de fecha / hora.
  - **Texto**. Por defecto, todos los valores de los atributos están indexados como cadenas o texto.
  - **Numérico**. Si el valor se puede convertir al tipo double, entonces se añade como índice atributo numérico.
  - **Fecha/Hora**. Si el valor se puede convertir al tipo date-time, entonces se añade como índice date/time.
- **Índices de contenido simple**. Dado un nombre de elemento y un valor, este índice devuelve todos los elementos que tienen un contenido simple correspondiente a este valor.
- **Índices Full-text**. Dada una palabra, este índice devuelve todos los nodos de texto que contienen una ocurrencia de esta palabra.

Para utilizar índices diferentes a los predeterminados, habrá que establecer lo que se denomina una especificación de indexado. Esto lo vamos a ver en el siguiente apartado

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Para que Qizx haga uso de índices elemento es necesario crearlos explícitamente.

Verdadero. ○ Falso. ○

Para saber más

Si quieres profundizar en las posibilidades de indexación de Qizx, consulta el siguiente enlace.

Información sobre Indexación en Qizx.

6.1.- Especificaciones de indexado.

Podemos asociar una especificación de indexado a una BD XML, de manera que ésta, se aplicará a todos sus documentos. **Una especificación de indexado puede contener**:

- Valores para parámetros generales.
- Especificaciones para índices `full-text`.
- Reglas para el reconocimiento de valores numéricos y de fecha en el contenido de elementos y atributos.

Pero, ¿qué es realmente una especificación de indexado? **Una especificación de indexado es un documento XML con las siguiente estructura**:

- El elemento raíz tiene el nombre `indexing`.
- El elemento `indexing` contiene los atributos que definen las condiciones generales.
- Contiene una lista de reglas aplicables a los elementos o atributos.

En el siguiente enlace puedes ver un ejemplo de especificación de indexado:

Estructura general de una especificación de índices.

Qizx proporciona una especificación de indexado por defecto, que se puede editar con QizxStudio y personalizar.

Si creamos una especificación de indexado nueva para una BD:

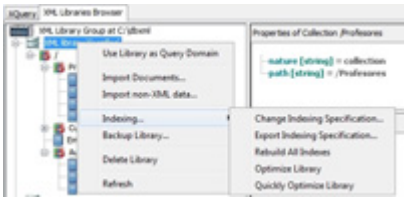
- Habrá que guardar el documento XML de la especificación junto a la BD o Library, de manera que ésta se iniciará al crear la BD, utilizándose de forma automática cuando se añaden documentos.
- Se deben incluir en ella las reglas que pone Qizx en su especificación por defecto, ya que en otro caso, éstas se perderán

Debes conocer

Las reglas de especificación de indexado por defecto de Qizx las puedes ver en el siguiente enlace:

Especificación de indexado por defecto de Qizx.

En la imagen ampliable de la derecha, puedes ver las diferentes opciones que proporciona QizxStudio para gestionar índices en una BD:



- Modificar la especificación de indexado actual.
- Exportar la especificación de indexado a un fichero `.xml`.
- Reconstruir los índices.
- Optimizar la BD, esto es, optimiza su almacenamiento y los índices de la BD.
- Optimizar de forma rápida la BD, esto es, en un tiempo limitado.

Autoevaluación

Señala las opciones correctas. Una especificación de indexado puede contener:

- [ ] Especificaciones para índices Full-text.
- [ ] Valores para parámetros generales.

- [ ] Un elemento index.
- [ ] Un documento XML.

Mostrar Información

6.2.- Re-Indexar una BD desde Java.

Para reindexar una BD XML de Qizx será necesario un objeto de la clase **`com.qizx.api.Indexing`**.

- Cada objeto `Indexing` es una especificación de indexado con las reglas y parámetros necesarios para construir índices en una BD o `Library`.
- El método `Indexing.parse(InputSource source)` analiza la especificación de indexado contenida en el texto XML del parámetro `source`.

Además se necesitan los siguientes **métodos de `Library`:**

- `Library.setIndexing(Indexing ind)`. Define las reglas de indexado para esa BD o `Library`, especificadas en el parámetro `ind`. Normalmente va seguida de un reindexado, que se realiza mediante el método `reIndex()`.
- `Library.reIndex()`. Reconstruye y reorganiza los índices de una BD

En el siguiente enlace tienes un ejemplo de reindexado de la BD Tutorial. Las especificaciones de indexado están escritas en el archivo **`indexing.xml`** que viene con la distribución Qizx Free Engine Edition y que encontrarás en la carpeta de instalación dentro del directorio **`docs\samples\programming\reindex\`**.

Ejemplo de indexación de una BD XML desde Java.

Autoevaluación

Señala si la siguiente afirmación es verdadera o falsa.

Mediante el método Library.setIndexing() se definen las reglas de indexado para una Library.

Verdadero. ○ Falso. ○

Para saber más

¿Cómo escribir consultas más eficientes en Qizx? El siguiente enlace te da ejemplos de cómo hacerlo.

Construir consultas más eficiente en Qizx.

Citas para pensar

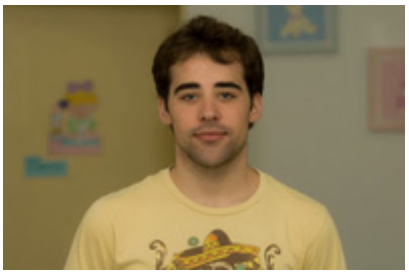
"Vale más saber alguna cosa de todo, que saberlo todo de una sola cosa."

Blaise Pascal

7.- Gestión de transacciones.

Caso práctico

Esta mañana, **Ada** le ha pedido a **Antonio** que pasara a ver la marcha del proyecto de **Ana**, **Juan** y **María**, a ver si se animaba a estudiar un CF. Ada sigue insistiendo en que su amigo debe estudiar un Ciclo Formativo de Informática. Al entrar a la sala de programación, **Antonio** ha escuchado decir a **Ana** —¿Habéis visto que Qizx debe confirmar las transacciones después de cada actualización? —**Antonio** saluda y dice— Esa me la sé, ¿no son las transacciones lo que se gestiona con commit y rollback? —**Ana**, Juan y María sonríen y lo saludan.



En muchos de los ejemplos que has visto, ha estado presente el concepto de **transacción:** conjunto de sentencias que se ejecutan formando una unidad de trabajo, de forma indivisible, de manera que se ejecutan todas o no se ejecuta ninguna.

Mediante la gestión de transacciones, los gestores XML proporcionan un acceso concurrente a los datos almacenados, mantienen la integridad y seguridad de los datos, y proporcionan un mecanismo de recuperación de la base de datos ante fallos.

Qizx soporta transacciones que cumplen el criterio ACID:

- **Atomicidad**: Se deben cumplir todas las operaciones de la transacción o no se cumple ninguna; no puede quedar a medias.
- **Consistencia**: La transacción solo termina si la base de datos queda en un estado consistente.
- **Isolation** (Aislamiento): Las transacciones sobre la misma información deben ser independientes, para que no interfieran sus operaciones y no se produzca ningún tipo de error.
- **Durabilidad**. Cuando la transacción termina el resultado de la misma perdura, y no se puede deshacer aunque falle el sistema.

Citas para pensar

"Cada día sabemos más y entendemos menos. "

Albert Einstein

7.1.- Los métodos commit() y rollback().

Los métodos para gestionar transacciones en Qizx los proporciona la interface **`Library`** y son:

- **`commit()`**. Confirma la transacción actual. Si este método se completa sin errores, las actualizaciones realizadas en la transacción garantizan la persistencia, y se hacen visibles a otras sesiones.
  - Una vez confirmada, desbloquea todos los objetos de la BD, previamente bloqueados.
  - Finalizada la transacción, el estado de la BD también refleja los cambios realizados posiblemente por otras transacciones.
- **`rollback()`**. Cancela la transacción actual.
  - Desbloquea todos los objetos de la BD, previamente bloqueados.
  - Después de un `rollback()` se refresca o actualiza el estado de la BD, de manera que se reflejan los cambios realizados por otras transacciones.

En Qizx, todas las operaciones de actualización sobre una BD o **`Library`**, para que tengan efecto permanente, deben ir seguidas de una confirmación o **`commit()`**. Por ejemplo, después de realizar una consulta de actualización, para que permanezcan los cambios es necesario realizar un **`commit()`**, tal y como refleja el siguiente código.

```
private static void evaluaExpresion(Expression expr, Library library)
        throws QizxException {
  /*ItemSequence result **/
  ItemSequence evaluate = expr.evaluate();
  // confirmar cambios en BD
  library.commit();
}
```

Código Java ejemplo de commit(). (0.01 MB)

Por otra parte, al cerrar una conexión a una BD es necesario comprobar si hay alguna transacción en curso, de manera que, si hay alguna en curso, para poder cerrar la BD habrá que cancelar la transacción actual.

El código que refleja de forma clara este hecho es el siguiente:

```
//Si la base de datos está inmersa en una transacción
if (bd.isModified()) {
  //deshace los cambios realizados por la transacción
  bd.rollback();
}
//cierra la conexión a la base de datos bd
bd.close();
```

Código Java ejemplo de rollback(). (0.01 MB)

8.- Tratamiento de excepciones.

Caso práctico

Ada se ha vuelto a reunir con **María** y **Juan**. Les ha insistido en que, como siempre, no deben descuidar la robustez de la aplicación. Ya saben que los clientes son muy exigentes y que una aplicación que falla de manera abrupta y se bloquea, no es el sello de la empresa BK Programación.

Juan y **María**, al unísono le han contestado —Tranquila **Ada**, la aplicación va a capturar y procesar todas las posibles excepciones. No habrá error que se le resista.

Una excepción es un suceso excepcional que ocurre durante la ejecución de un programa y que interrumpe su ejecución o flujo normal de sentencias. El origen o causa de una excepción es un error en el programa.

Gestionando de manera adecuada las posibles excepciones del programa, podemos construir programas robustos, que pueden continuar su ejecución sin que el problema les afecte, o bien finalizar de manera no abrupta.

En Qizx, la clase que contiene los diferentes tipos de excecpiones es `QizxException` del paquete `com.qizx.api`. Esta clase extiende a Exception, superclase de todas las excepciones del API y sus constructores son:

- `QizxException(String message)`. Construye un mensaje con la razón o causa de excepción. El código del error es indefinido.
- `QizxException(String message, Throwable cause)`. Construye un mensaje con la razón o causa de excepción, y la excepción. El código del error es indefinido.
- `QizxException(QName errorCode, String message)`. Construye un mensaje con la razón de la excepción y un código de error XQuery.

En el siguiente segmento de código, puedes ver el ejemplo de un método que puede generar excepciones que deben ser capturadas. En este caso, se utiliza **`throws`**, de manera que si se produce un error en la ejecución del método, error que puede ser originado al intentar cerrar la conexión de un objeto **`Library`** o bien todas las **`Library`** de un grupo, el método no se encarga de resolverlo, sino que deja el tratamiento de la excepción al bloque desde donde se hace la llamada.

Código Java ejemplo de QizxException. (0.01 MB)

Tres subclases directas de **`QizxExcepction`** son:

- `CompilationException`. Excepciones debidas a errores en la compilación de consultas a la BD.
- `DataModelException`. Excepciones debidas a manipulación de documentos XML en la BD.
- `EvaluationException`. Excepciones debidas a errores en la ejecución de una consulta.

En el siguiente apartado verás un ejemplo de captura de excepciones **`CompilationException`**.

Para saber más

Desde este enlace puede consultar un artículo sobre el tratamiento de excecpciones en Java.

Tratamiento de excepciones en Java.

8.1.- Excepciones en el procesamiento de consultas.

Cuando se procesa una consulta XQuery o una consulta de actualización XQuery, pueden ocurrir errores en el proceso de compilación y evaluación de la expresión de consulta, que suponen el lanzamiento de excepciones.

Estas excepciones las puedes gestionar mediante dos subclases directas de la clase **`QizxException`**:

- `CompilationException`. Se lanza cuando hay errores de compilación en una expresión de consulta. Los mensajes asociados pueden ser: el error, una advertencia o warning, y la descripción o detalle del error. Podemos gestionar esos errores mediante los siguientes métodos
  - `getErrorCount()` devuelve el número actual de errores.
  - `getMessages()` devuelve una lista de errores o warnings
- `EvaluationException`. Se lanza cuando hay errores en la ejecución de una expresión XQuery.

Por ejemplo, el siguiente código muestra un método que compila una expresión de consulta y controla posibles errores, almacenando en un **`array`** de tipo **`Message`** los posibles errores de compilación, Observa que en este caso, el tratamiento de la excepción se realiza mediante un bloque **`try-catch`**, y que en caso de que se produzca se captura la lista de errores.

```
//Método para compilar la consulta controlando errores
 private static Expression compileExpression(Library bd,
        String consultaXquery)
        throws IOException, QizxException {
   Expression expr;
   try {
     expr = bd.compileExpression(consultaXquery);
   } catch (CompilationException e) {
    //mensajes de error devueltos tras la compilación
     Message[] messages = e.getMessages();
     for (int i = 0; i < messages.length; ++i) {
       System.out.println(messages[i].toString());
     }
     throw e;
   }
//devuelve consulta compilada
   return expr;
 }
```

Código Java ejemplo de CompillationException. (0.01 MB)

Para saber más

En el siguiente enlace puede ver un ejemplo de tratamiento de excepciones con Qizx en un entorno Web.

Ejemplo de tratamiento de excepciones con Qizx en entorno Web.

Anexo.- Licencias de recursos.

Licencias de recursos utilizados en la Unidad de Trabajo.

Recurso (1)	Datos del recurso (1)	Recurso (2)	Datos del recurso (2
	Autoría: Isabel M. Cruz Granados. Licencia: Uso Educativo-nc. Procedencia: Captura de pantalla del programa QizxStudio de Qizx Free Engine Edition.		Autoría: Isabel M. (Granados. Licencia: Uso Educativo-r Procedencia: Captura pantalla del progr QizxStudio de Qizx I Engine Edition.
	Autoría: Isabel M. Cruz Granados. Licencia: Uso Educativo-nc. Procedencia: Captura de pantalla del programa QizxStudio de Qizx Free Engine Edition.		Autoría: Isabel M. (Granados. Licencia: Uso Educativo-r Procedencia: Captura pantalla del progr QizxStudio de Qizx I Engine Edition
	Autoría: Emma Gracia Lor. Licencia: CC-by-nc-sa. Procedencia: http://www.flickr.com/photos/paideiaeducacion/5102909303/		Autoría: fsse8info. Licencia: CC BY-SA. Procedencia: ht www.flickr.com/photos/fss info/4194980532/
	Autoría: Howard Stanbury. Licencia: CC BY-NC-SA. Procedencia: http://www.flickr.com/photos/stanbury/5852623652/		Autoría: Justin See. Licencia: CC BY. Procedencia: ht www.flickr.com/photos/ koalazymonkey/3651288
	Autoría: Adam Prince. Licencia: CC BY-NC-SA. Procedencia: http://www.flickr.com/photos/adam_prince/3908404628/		Autoría: Tantek Çelik. Licencia: CC BY-NC. Procedencia: ht www.flickr.com/photos/ tantek/539497686/